

JAVA INTERVIEW PREPARATION

CHAPTER 18

CONCURRENTHASHMAP INTERNALS

Senior / Lead Java Developer Textbook Edition

Full reading material - not summarized

ConcurrentHashMap Internals: Atomic Updates Without a Global Map Lock

Senior / Lead Java Developer Reading Chapter

ConcurrentHashMap supports high-concurrency access while preserving defined atomic operations. It is not merely a HashMap with synchronized methods. Its design allows retrievals to proceed with very little coordination, updates to contend mainly around affected buckets, and resizing work to be shared. The class solves data-structure synchronization, but it cannot automatically make a multi-step business workflow atomic.

Why a Synchronized HashMap Is Different

Wrapping a map with `Collections.synchronizedMap` places a common synchronization boundary around its methods. This can be correct, but unrelated keys contend for the same lock and compound iteration still requires the documented external synchronization.

```
Map<String, Session> sessions =
    Collections.synchronizedMap(new HashMap<>());

synchronized (sessions) {
    for (Session session : sessions.values()) {
        inspect(session);
    }
}
```

ConcurrentHashMap is designed for concurrent operations rather than global mutual exclusion. Retrievals generally do not acquire bin locks. Updates coordinate using compare-and-set operations and small synchronization regions associated with bins or table-management state.

Conceptual Structure in Modern JDKs

Modern implementations use a bucket table containing nodes. A bucket may be empty, contain a list, contain a tree, or temporarily contain a forwarding marker during resize.

```
table
[0] -> node -> node
[1] -> empty
[2] -> tree bin
[3] -> forwarding node ----> next table during resize
```

Older Java versions used a visible segment architecture. Describing current ConcurrentHashMap as a fixed collection of independently locked segments is outdated. Constructors may retain a `concurrencyLevel` parameter for compatibility and sizing guidance, but modern locking is not the old segment design.

Reads, Visibility, and Happens-Before

A successful retrieval of a value for a key reflects a happens-before relationship with the completed insertion or update that established that value. Internal volatile reads and publication rules allow readers to observe safely published nodes and values.

```
configuration.put("pricing", fullyInitializedConfig);

// A thread that later observes this mapping also observes the
// initialization actions that happened before the completed put.
Configuration config = configuration.get("pricing");
```

This guarantee applies to the map publication action. If the value is mutable and is later changed without synchronization, `ConcurrentHashMap` does not make those later fields magically thread-safe.

Update Mechanics

For an empty bucket, insertion can often install a node with compare-and-set. For a populated bucket, an updater coordinates on that bin while locating or appending the entry. Therefore two threads updating different buckets can often proceed independently.

```
Thread A -> bucket 2 ---- lock/coordinate here
Thread B -> bucket 9 ---- lock/coordinate independently
Thread C -> bucket 2 ---- may contend with A
```

Collision quality still matters. Many keys concentrated into one bin reduce concurrency as well as lookup performance. Tree bins control lookup degradation but cannot eliminate contention on a hot logical key.

Atomic Map Operations

The most important API feature is not internal locking; it is atomic compound operations.

```
cache.putIfAbsent(key, createValue());
cache.computeIfAbsent(key, this::loadValue);
cache.computeIfPresent(key, (k, old) -> refresh(old));
cache.compute(key, (k, old) -> calculate(k, old));
cache.merge(key, delta, Integer::sum);
cache.replace(key, expectedValue, newValue);
cache.remove(key, expectedValue);
```

These express intent inside the map's synchronization boundary. A manual `containsKey` followed by `put` has a race: multiple threads can all observe absence and create competing values.

computeIfAbsent Is Not a General Transaction

The mapping function should be short, deterministic enough for the application, and free of recursive updates to the same map that can violate the operation's rules. It must not return null when a mapping is required.

```
Customer customer = customers.computeIfAbsent(
    customerId,
    id -> repository.load(id)
);
```

This prevents competing successful installations for the same key, but it does not turn the repository call into a distributed transaction. Slow I/O in mapping functions can create contention for callers interested in the same mapping. Failure, retry, stale data, eviction, timeout, and cancellation still require a cache policy.

Do not assume the function executes exactly once across all time. A failed computation installs nothing; removal can allow a later computation; implementation and races can cause more attempts than a business side effect can tolerate. Keep externally visible side effects idempotent or separate them from map population.

Nulls Are Deliberately Forbidden

ConcurrentHashMap rejects null keys and values. In a concurrent map, null from get must unambiguously mean that no mapping was observed. Allowing stored null values would make absence and presence difficult to distinguish during concurrent changes.

```
Value value = map.get(key);
if (value == null) {
    // no mapping was observed at this point
}
```

This is an observation, not a permanent fact. Another thread may add the key immediately afterward.

Size Is a Moving Observation

Under concurrent mutation, size, isEmpty, and containsValue describe a transient aggregate. They are useful for monitoring and approximate decisions, not for establishing a global invariant.

```
if (map.size() < limit) {
    map.put(key, value); // does not atomically enforce the limit
}
```

Use a Semaphore, bounded cache, explicit lock, database constraint, or another structure designed to enforce the actual limit. A concurrent collection makes individual operations safe; it does not freeze the world between them.

Cooperative Resizing

The table grows when population and bin distribution require more capacity. Resizing is expensive, so modern ConcurrentHashMap allows multiple updating threads to help transfer bins to the new table. Forwarding nodes tell readers and writers that a bin has moved.

```
old table          new table
bucket 4 --moved-----> buckets 4 and 20
bucket 5 --worker A----> ...
bucket 6 --worker B----> ...
```

Correct initial sizing still matters for very large maps. It reduces transfer work and temporary memory pressure, though premature massive sizing wastes memory and iteration locality.

Weakly Consistent Iteration

Iterators do not throw ConcurrentModificationException merely because another thread updates the map. They are weakly consistent: they can reflect some modifications during traversal, never require a single global snapshot, and do not return structurally corrupt data.

This property supports monitoring and maintenance operations that tolerate an evolving view. It is not suitable when a report requires an exact point-in-time state. For that, create a snapshot under an application-defined consistency boundary or read from a versioned source.

Counters with LongAdder

A common pattern stores LongAdder values for frequently updated statistics.

```
ConcurrentHashMap<String, LongAdder> counts = new ConcurrentHashMap<>();

counts.computeIfAbsent(eventType, ignored -> new LongAdder())
    .increment();
```

LongAdder spreads highly contended updates across internal cells and combines them when sum is requested. Its sum is not an atomic transaction with concurrent increments, so it suits telemetry and frequency maps better than financial balances or strict limits.

Per-Key Atomicity Versus Cross-Key Invariants

Atomic methods protect operations associated with the relevant mapping. They do not atomically coordinate an invariant across multiple keys.

```
inventory.compute(productA, reserveOne);
inventory.compute(productB, reserveOne);
```

If both reservations must succeed or neither may succeed, two compute calls are insufficient. Use a higher-level lock with stable ordering, a transaction in the authoritative database, an actor/partition model, or redesign the aggregate boundary.

Similarly, storing a mutable ArrayList as a value is unsafe unless access to that list is separately controlled. The map safely publishes and locates the reference, not every operation on the referenced object.

Bulk Operations

ConcurrentHashMap provides parallel-capable forEach, search, and reduce operations with a parallelism threshold. These operations observe a concurrently changing map and require side-effect-safe functions.

Parallel execution is not automatically faster. Map size, function cost, common-pool contention, CPU availability, and latency requirements determine whether it helps. Use a high threshold or sequential operation for small maps and measure realistic workloads.

Production Failure Patterns

Frequent failures include an unbounded map used as a cache, a slow computeIfAbsent loader that concentrates requests on hot keys, mutable values changed unsafely, and an assumed global invariant built from size plus put. A hash collision hotspot can produce bin contention. Metrics code can also mistake weakly consistent totals for exact accounting.

Diagnose key cardinality, retained value size, hot-key distribution, mapping-function latency, collision behavior, resize allocation, blocked threads, and the thread safety of value objects. A concurrency-safe container is only one layer of a concurrency-safe design.

Interview-Ready Summary

Modern ConcurrentHashMap uses a bucket table with CAS, bin-level coordination, tree bins, and cooperative resizing rather than the old fixed-segment architecture. Reads are generally non-blocking and completed updates safely publish their mappings.

Use putIfAbsent, compute, merge, replace, and conditional remove for atomic map operations. Nulls are forbidden so null lookup has clear absence meaning. Iterators are weakly consistent rather than fail-fast and do not form snapshots.

Atomicity is normally scoped to a mapping, not multiple keys or mutable objects stored as values. Long-running computations, unbounded retention, hot keys, and downstream side effects remain application

concerns.

Revision Notes

- ConcurrentHashMap is not a HashMap protected by one global lock.
- The old segment explanation does not describe modern implementations.
- Reads generally proceed without bin locks.
- Empty-bin insertion can use compare-and-set.
- Updates to populated bins coordinate locally.
- Poor hash distribution also reduces update concurrency.
- Completed insertion safely publishes the stored reference.
- Later mutation of a value still needs its own synchronization.
- Prefer atomic APIs over manual check-then-act sequences.
- `computeIfAbsent` is map atomicity, not a distributed transaction.
- Keep mapping functions short and side effects controlled.
- Null keys and values are forbidden.
- Aggregate size under mutation is a moving observation.
- Cooperative resizing lets multiple threads help transfer bins.
- Weakly consistent iteration is not a point-in-time snapshot.
- `LongAdder` suits high-contention approximate statistics.
- Per-key atomicity does not enforce cross-key invariants.
- Thread-safe maps can contain thread-unsafe values.
- Unbounded ConcurrentHashMap caches still leak by retention policy.
- Diagnose hot keys, loader latency, value safety, resizing, and cardinality.